LEAD AI ENGINEER • PART 02 / 12

LLM & Generative AI Engineering

10 exact scenario questions

10 practical, interview-ready answers

- Root-cause diagnosis and production solutions
- Interview follow-up questions and technical trade-offs
- Quick memory shortcuts and common mistakes

Created by Ramachandran

LLM & Generative AI Engineering

10 scenarios to master

01 Your LLM generates different answers for the same question. How would you improve consistency?

02 An LLM hallucinates policy clauses even though the correct documents are available. How would you troubleshoot?

03 Your prompt performs well on 90% of cases but fails on complex queries. How would you systematically improve it?

04 Your AI application spends INR 5 lakh per month on tokens. How would you reduce cost without significantly degrading quality?

05 GPT-based responses take 15 seconds. How would you optimize latency?

06 Your model produces invalid JSON in production. How would you enforce reliable structured outputs?

07 A new model version improves quality but doubles inference cost. How would you decide whether to migrate?

08 Users ask multilingual questions but the LLM performs poorly in some languages. What would you change?

09 A customer wants to fine-tune an LLM instead of using RAG. How would you evaluate the decision?

10 Your primary LLM provider becomes unavailable. How would you design model failover and graceful degradation?

QUESTION 01

Your LLM generates different answers for the same question. How would you improve consistency?

1. DIAGNOSE THE PROBLEM

- Confirm what must remain stable: facts, wording, JSON schema, or business decision.
- Replay identical inputs with model ID, prompt version, context, sampling settings, and tool responses fixed.
- Separate model variability from changing retrieval results, time-dependent tools, and conversation history.

2. PRODUCTION-READY SOLUTION

- Use low temperature where supported, explicit decision rules, constrained output schemas, and few-shot edge cases.
- Pin model/prompt versions; normalize retrieved context ordering; use deterministic post-processing for business rules.
- Run repeated trials on a golden dataset and compare semantic correctness, format compliance, and variance.

PRACTICAL EXAMPLE / CHECK

Replay: same prompt + model + context + tools; evaluate 20 runs per case.

INTERVIEW FOLLOW-UP: Does temperature=0 guarantee identical output? No; backend and model behavior can still vary.

QUICK MEMORY: PIN → CONSTRAIN → REPLAY → MEASURE

AVOID: Promising bit-for-bit determinism from temperature=0 alone.

QUESTION 02

An LLM hallucinates policy clauses even though the correct documents are available. How would you troubleshoot?

1. DIAGNOSE THE PROBLEM

- Inspect the retrieved evidence and exact citations: was the correct clause present in the final LLM context?
- Classify failure as retrieval miss, context truncation, conflicting versions, prompt issue, or unsupported synthesis.
- Check whether the answer introduced obligations, dates, or parties absent from the source.

2. PRODUCTION-READY SOLUTION

- Enforce document/version/ACL filters and improve retrieval/reranking before changing the generator.
- Require clause-level evidence and abstention when no supported clause exists; separate quotes from interpretation.
- Evaluate claim-level groundedness and citation support with human-reviewed policy test cases.

PRACTICAL EXAMPLE / CHECK

Test: 30 policy questions × supported / missing / conflicting clauses.

INTERVIEW FOLLOW-UP: What if the relevant document exists but is excluded by authorization? Never bypass access controls.

QUICK MEMORY: RETRIEVE → VERIFY → GROUND → ABSTAIN

AVOID: Adding “do not hallucinate” to the prompt and treating the issue as solved.

QUESTION 03

Your prompt performs well on 90% of cases but fails on complex queries. How would you systematically improve it?

1. DIAGNOSE THE PROBLEM

- Build a failure taxonomy: multi-hop, long context, ambiguity, calculation, language, formatting, or tool use.
- Slice evaluation by query type, length, tenant, and document characteristics; inspect representative traces.
- Determine whether the issue originates in retrieval, reasoning, instruction conflict, or missing tools.

2. PRODUCTION-READY SOLUTION

- Create labeled regression cases and targeted examples for each failure category.
- Use query decomposition or tool workflows only where evidence shows they help; improve context selection.
- Run prompt A/B tests with quality, latency, cost, and regression gates before rollout.

PRACTICAL EXAMPLE / CHECK

Track per-slice pass rate; do not hide 0% multi-hop accuracy inside 90% overall.

INTERVIEW FOLLOW-UP: How do you prevent overfitting to 10 failed prompts? Use holdout cases and diverse real traffic.

QUICK MEMORY: CLUSTER → HYPOTHESIZE → TEST → REGRESS

AVOID: Continuously lengthening the system prompt without measuring regressions.

QUESTION 04

Your AI application spends INR 5 lakh per month on tokens. How would you reduce cost without significantly degrading quality?

1. DIAGNOSE THE PROBLEM

- Break cost into requests × input tokens × output tokens × model price; include retries and tool calls.
- Segment spend by feature, tenant, prompt version, context size, and task complexity.
- Identify wasted retrieval context, repeated queries, verbose responses, and runaway agents.

2. PRODUCTION-READY SOLUTION

- Route simple tasks to smaller models after quality testing; trim/rerank context and cap output tokens.
- Cache safe reusable results or prompt prefixes where supported; batch offline workloads when suitable.
- Apply quotas, per-request budgets, retry caps, and cost dashboards; protect high-value workflows.

PRACTICAL EXAMPLE / CHECK

Example: 30% fewer tokens × same request volume ≈ 30% token-usage reduction, not guaranteed bill savings.

INTERVIEW FOLLOW-UP: Which quality metric must not regress? Define task-specific accuracy and business acceptance thresholds.

QUICK MEMORY: MEASURE → ROUTE → TRIM → CACHE → GOVERN

AVOID: Switching every request to the cheapest model without quality validation.

QUESTION 05

GPT-based responses take 15 seconds. How would you optimize latency?

1. DIAGNOSE THE PROBLEM

- Trace end-to-end: queue, retrieval, rerank, prompt assembly, provider time-to-first-token, generation, retries.
- Inspect input/output token counts, provider rate limits, region, network, and concurrency saturation.
- Compare p50 and p95 by request class; distinguish first-token delay from total generation time.

2. PRODUCTION-READY SOLUTION

- Reduce unnecessary context/output length; choose a faster model only after task-quality comparison.
- Stream output for perceived responsiveness; parallelize independent I/O without overwhelming dependencies.
- Tune concurrency, caching, connection reuse, and retry backoff; set latency budgets per stage.

PRACTICAL EXAMPLE / CHECK

Latency budget example: retrieval 0.4s + rerank 0.3s + model 2.8s + overhead 0.5s = 4.0s.

INTERVIEW FOLLOW-UP: Does streaming lower total inference time? Not necessarily; it usually improves perceived latency.

QUICK MEMORY: TRACE → TOKENS → QUEUES → OPTIMIZE → LOAD TEST

AVOID: Adding API replicas when the LLM provider is the bottleneck.

QUESTION 06

Your model produces invalid JSON in production. How would you enforce reliable structured outputs?

1. DIAGNOSE THE PROBLEM

- Collect invalid responses and classify syntax errors, schema violations, truncated output, and unsafe values.
- Check token caps, stop sequences, streaming assembly, and whether the selected model supports schema enforcement.
- Quantify invalid-rate by model and prompt version, not just anecdotal failures.

2. PRODUCTION-READY SOLUTION

- Prefer provider-supported strict structured outputs / JSON Schema when available.
- Validate with Pydantic; enforce enums, types, lengths, and business rules server-side.
- Use bounded repair/retry for recoverable errors and explicit fallback for persistent failures.

PRACTICAL EXAMPLE / CHECK

Pydantic: Result.model_validate_json(response_text); catch ValidationError.

INTERVIEW FOLLOW-UP: Is syntactically valid JSON enough? No; semantic and authorization validation still matter.

QUICK MEMORY: SCHEMA → VALIDATE → RETRY → FALLBACK

AVOID: Using regex cleanup as the primary correctness mechanism.

QUESTION 07

A new model version improves quality but doubles inference cost. How would you decide whether to migrate?

1. DIAGNOSE THE PROBLEM

- Define business-weighted quality: factuality, safety, task success, latency, cost, and throughput.
- Benchmark old and new models on a representative golden set plus hard edge cases.
- Check migration differences in tool calls, structured outputs, multilingual behavior, and prompts.

2. PRODUCTION-READY SOLUTION

- Calculate incremental cost per successful task, not cost per token alone.
- Canary by tenant or traffic share with rollback thresholds; monitor real user outcomes.
- Use task-based routing: premium model only for workflows where the uplift justifies expense.

PRACTICAL EXAMPLE / CHECK

Decision: (value of extra successful tasks − added inference cost − operating cost) > 0.

INTERVIEW FOLLOW-UP: What if quality rises 5% but cost doubles? Evaluate business value, workload mix, and alternatives.

QUICK MEMORY: BENCHMARK → VALUE → CANARY → ROUTE

AVOID: Migrating because a public benchmark ranks the new model higher.

QUESTION 08

Users ask multilingual questions but the LLM performs poorly in some languages. What would you change?

1. DIAGNOSE THE PROBLEM

- Measure quality separately by language, script, domain terms, and code-switching patterns.
- Check OCR/extraction, chunking, embeddings, retrieval, and generation; errors may precede the LLM.
- Use native-speaker-reviewed samples to detect meaning drift and culturally incorrect terminology.

2. PRODUCTION-READY SOLUTION

- Select multilingual embeddings and language-aware retrieval/reranking where validated.
- Preserve source language where possible; use translation pivot only after accuracy tests.
- Provide terminology glossaries, language-specific evaluation sets, and locale-aware response templates.

PRACTICAL EXAMPLE / CHECK

Eval matrix: language × document type × query type × groundedness / retrieval recall.

INTERVIEW FOLLOW-UP: Would you translate everything to English? Only if tests show improved quality and acceptable risk.

QUICK MEMORY: EXTRACT → RETRIEVE → GENERATE → EVALUATE BY LANGUAGE

AVOID: Assuming a multilingual chat model fixes monolingual retrieval.

QUESTION 09

A customer wants to fine-tune an LLM instead of using RAG. How would you evaluate the decision?

1. DIAGNOSE THE PROBLEM

- Clarify the objective: new knowledge, domain style, output format, specialized behavior, or lower inference cost.
- Check update frequency, access controls, citations, training examples, and maintenance budget.
- Build a RAG baseline and compare with prompting, fine-tuning, or hybrid approaches.

2. PRODUCTION-READY SOLUTION

- Use RAG for changing/private source facts and traceable citations; use fine-tuning for learned task behavior.
- Fine-tune only with licensed, clean, representative examples and separate holdout evaluation.
- Consider hybrid: fine-tuned behavior plus retrieval of current authorized documents.

PRACTICAL EXAMPLE / CHECK

Decision grid: freshness / citations / behavior change / dataset size / operating cost.

INTERVIEW FOLLOW-UP: Can fine-tuning replace retrieval for frequently updated contracts? Usually not reliably.

QUICK MEMORY: FACTS → RAG | BEHAVIOR → FINE-TUNE | BOTH → HYBRID

AVOID: Claiming fine-tuning automatically gives accurate, up-to-date enterprise knowledge.

QUESTION 10

Your primary LLM provider becomes unavailable. How would you design model failover and graceful degradation?

1. DIAGNOSE THE PROBLEM

- Define failure modes: timeouts, 429 throttling, regional outage, auth failure, and degraded output quality.
- Check dependency SLAs, per-model limits, data residency, and feature/schema compatibility.
- Identify which workflows can degrade safely and which must stop.

2. PRODUCTION-READY SOLUTION

- Use circuit breakers, timeouts, jittered retries, and bounded failover to tested alternate deployments.
- Normalize provider interfaces; verify tool/JSON compatibility, privacy controls, and prompt behavior.
- Offer graceful degradation (queued processing, read-only answers, or explicit unavailability) and alert operators.

PRACTICAL EXAMPLE / CHECK

Fallback rule: primary timeout → breaker opens → eligible secondary → validate output.

INTERVIEW FOLLOW-UP: What if backup provider violates residency rules? Do not route restricted data there.

QUICK MEMORY: DETECT → ISOLATE → FAIL OVER → VALIDATE → RECOVER

AVOID: Blind retries across providers without idempotency, cost, or data-governance controls.